



TASK LIST: Sinkronisasi API ↔ App Agrotani

Hasil audit menyeluruh terhadap seluruh source code Backend (NestJS) dan Frontend (Flutter).
Setiap task berisi: masalah, akar penyebab, keputusan solusi, dan sisi mana yang perlu diubah.

Legenda: ● Critical · ● Moderate · ● Minor

Sisi: [BE] = Backend · [FE] = Frontend · [BOTH] = Keduanya

Daftar Isi

- [A. Auth — Login/Register Mismatch](#)
- [B. Scan — Confidence & Parsing](#)
- [C. Chat — History & Session](#)
- [D. Profile & Logout](#)
- [E. Security & Backend Hygiene](#)
- [F. Minor / Polish](#)

A. Auth — Login/Register Mismatch

A1. ● Login: Frontend kirim "Username" tapi Backend expects email @IsEmail()

Masalah:

- FE LoginScreen punya field berlabel **"Username"**, nilai dikirim sebagai parameter `email` ke `POST /api/auth/login`
- BE `UserLoginDto` memvalidasi field `email` dengan `@IsEmail()` — artinya hanya format email yang diterima
- Dev hint auto-fills `budi@agrotani.id` (format email) jadi saat testing "berhasil", tapi kalau user ketik username biasa (misal `budi`) → **400 Bad Request**

Akar masalah: Ketidaksepakatan apakah identifier login itu email atau username.

Keputusan:

Gunakan email sebagai identifier login. Alasannya:

- Backend sudah benar memvalidasi email
- Database schema sudah `email: @unique`
- Lebih standar dan scalable (bisa reset password via email nanti)

Task:

- ☐ [FE] **LoginScreen:** Ubah label field dari "Username" menjadi **"Email"**, ganti `keyboardType` ke `TextInputType.emailAddress`, tambah validasi format email di Flutter
- ☐ [FE] **LoginScreen:** Ubah hint text/dev autofill jadi jelas bahwa ini email

A2. ● Register: Frontend tidak mengirim email, tapi Backend WAJIB email

Masalah:

- FE RegisterScreen punya 7 field: Nama, No HP, Alamat, **Username**, Tentang, Password, Konfirmasi Password
- FE mengirim `{ name, phone, address, username, password, aboutMe }` — **TIDAK ADA field email**
- BE `UserRegistrationDto` mewajibkan `email` dengan `@IsEmail()`
- Entah di FE atau BE, email di-generate otomatis (user melaporkan ini terjadi)
- BE bahkan punya komentar: `username? → "Flutter mengirimnya, kita abaikan"`

Akar masalah: FE dan BE dibangun dengan asumsi berbeda tentang field registrasi.

Keputusan:

Hapus field Username di FE, ganti dengan field Email.

- Email = identifier utama (sesuai database schema yang sudah ada)
- Hapus auto-generate email
- Backend tetap apa adanya (sudah benar)

Task:

- ☐ [FE] **RegisterScreen**: Hapus field "Username", tambah field "**Email**" dengan validasi @IsEmail format
- ☐ [FE] **RegisterScreen**: Pastikan data yang dikirim ke API: { name, email, password, phone?, address?, aboutMe? }
- ☐ [FE] **auth_repository.dart**: Hapus logika auto-generate email, kirim email yang user ketik langsung
- ☐ [BE] **UserRegistrationDto**: Hapus field username? beserta komentar "kita abaikan"

A3. 🟡 Login: Nama user jadi nomor HP (LoginScreen bug)

Masalah (catatan dari audit awal — cek apakah masih relevan di versi saat ini):

- Pada versi awal Flutter, LoginScreen mengirim name: _phoneController.text dan phone: _phoneController.text — nama = nomor HP
- Ini mungkin sudah berubah di versi terbaru (yang pakai email+password), tapi perlu dicek

Task:

- ☐ [FE] Verifikasi bahwa login yang berhasil **tidak mengoverwrite nama user di database** — karena POST /auth/login di backend tidak update nama, ini harusnya aman. Konfirmasi.

A4. 🟡 UserModel FE punya field yang tidak dikirim BE / sebaliknya

Masalah:

- FE UserModel punya field: id, name, email, phone, address, aboutMe, createdAt
- BE getProfile return: id, name, email, phone, address, aboutMe, createdAt (select specific fields)
- **Match!** Tapi FE juga punya totalScans dan totalChats yang di-parse dari _count — ini TIDAK dikembalikan oleh BE getProfile

Keputusan:

Tambahkan _count di BE getProfile response ATAU hapus stats dari FE ProfileScreen (sudah hardcoded anyway — lihat Task D3).

Task:

- ☐ [BE] **auth.service.ts** → **getProfile()**: Tambahkan include: { _count: { select: { scans: true, chatSessions: true } } } di query Prisma agar FE dapat data stats yang benar
- ☐ [FE] **UserModel**: Pastikan fromJson membaca _count.scans dan _count.chatSessions dengan benar (cek apakah sudah ada atau perlu ditambah)

B. Scan — Confidence & Parsing

B1. 🔴 Gemini System Instruction vs Prompt Format: KONTRADIKTIF

Masalah:

- gemini.service.ts punya **system instruction** yang meminta output JSON:

```
{ "diagnosis": "...", "confidence": 0.85, "severity": "rendah|sedang|tinggi", ... }
```
- Tapi **prompt di analyzeImage()** meminta output markdown:

```
**🔍 Diagnosis:** ...
**📊 Tingkat Keparahan:** ...
**💡 Rekomendasi:** ...
**Tingkat keyakinan:** ...
```

- `scan.service.ts` parsing-nya mencari keyword per-line (expects markdown)
- Gemini **bingung** antara JSON atau markdown → output inconsistent → parsing sering gagal

Ini akar masalah utama kenapa confidence "Tidak diketahui" dan severity kadang tidak terisi.

Keputusan:

Pilih SATU format: Markdown. Alasannya:

- Parsing markdown lebih toleran terhadap variasi output
- JSON dari LLM sering malformed (missing quotes, trailing comma)
- Markdown lebih mudah di-debug (bisa langsung baca `rawResponse`)

Task:

- ☐ [BE] `gemini.service.ts` → **system instruction**: Hapus semua instruksi yang meminta JSON output. Ganti system instruction agar konsisten meminta output markdown dengan format yang jelas:

```
Selalu jawab dalam format markdown berikut:
**Diagnosis:** [nama penyakit]
**Keparahan:** [Ringan/Sedang/Parah]
**Keyakinan:** [angka]%
**Rekomendasi:**
- langkah 1
- langkah 2
**Catatan:** [catatan penting]
```

- ☐ [BE] `gemini.service.ts` → **analyzeImage() prompt**: Sinkronkan format prompt dengan system instruction di atas
- ☐ [BE] `gemini.service.ts` → **chat()**: Hapus instruksi JSON output untuk chat juga. Chat cukup return plain text.

B2. 🚫 Scan parsing fragile — confidence sering kosong

Masalah:

- `scan.service.ts` parsing mencari keyword lalu ambil baris berikutnya
- Kalau Gemini taruh jawaban di baris yang sama dengan keyword (misal `**Diagnosis:** Blas`) → parsing gagal
- Kalau Gemini pakai format sedikit berbeda → parsing gagal
- confidence kadang tidak ter-parse → di DB jadi kosong → FE tampil "0%" atau "Tidak diketahui"

Task:

- ☐ [BE] `scan.service.ts` → **parseResponse()**: Rewrite parser agar lebih robust:
 - Gunakan regex yang menangkap value **di baris yang sama** maupun **di baris berikutnya**
 - Pattern: `/\s*{0,2}(\?:Diagnosis|🔍)(?:\s*|)(.+?)(?=\n\s*{0,2}(\?:Keparahan|Tingkat|📊)|$)/is`
 - Untuk confidence: parse angka dari teks (cari pattern `(\d+)%` di seluruh response)
 - Untuk severity: normalize ke salah satu dari ["Ringan", "Sedang", "Parah"] — map variasi seperti "rendah" → "Ringan", "tinggi" → "Parah"
 - Fallback: kalau parsing gagal total, simpan keseluruhan response sebagai `diagnosis`, set `severity="Tidak diketahui"`, `confidence="0%"`

B3. 🟡 Severity matching di FE terlalu strict

Masalah:

- FE `DiagnosisCard` (atau `ScanResultScreen`) match severity dengan `switch` on lowercase: "ringan", "sedang", "parah"
- BE bisa mengirim severity multi-line atau dengan teks tambahan (misal "Sedang - perlu ditangani segera")

- Kalau tidak match → warna default grey, progress 0.0

Task:

- ☐ [FE] **ScanResultScreen / DiagnosisCard**: Ubah severity matching jadi `contains` bukan `exact match`:

```
if (severity.toLowerCase().contains('ringan')) → green, 0.33
if (severity.toLowerCase().contains('sedang')) → orange, 0.66
if (severity.toLowerCase().contains('parah')) → red, 1.0
```

- ☐ [BE] **scan.service.ts**: Setelah parsing severity, **normalize** hasilnya ke salah satu: "Ringan", "Sedang", atau "Parah" sebelum simpan ke DB. Ini lebih bersih daripada bergantung pada FE.

B4. 🟡 imageUrl di scan tidak bisa diakses dari Flutter

Masalah:

- BE simpan `imageUrl` sebagai path lokal (misal `uploads/abc123`)
- Tidak ada static file serving di `main.ts` (audit kedua menunjukkan ini belum ada)
- FE coba load image via `Image.network('${ApiConfig.baseUrl}${scanResult.imageUrl}')` → **gagal** karena route `/uploads/...` tidak di-serve

Task:

- ☐ [BE] **main.ts**: Tambahkan static file serving untuk folder uploads:

```
app.useStaticAssets(join(__dirname, '..', 'uploads'), { prefix: '/uploads' });
```

- ☐ [BE] **scan.service.ts**: Pastikan `imageUrl` disimpan sebagai `/uploads/filename.jpg` (dengan leading slash) agar konsisten

C. Chat — History & Session

C1. 🔴 Chat history tidak ter-load saat buka app

Masalah:

- FE `ChatNotifier.build()` return empty state — **tidak memanggil API untuk load session/history**
- Kalau user tutup app dan buka lagi → chat kosong, padahal di database masih ada
- Method `getSessions()` dan `getMessages()` ada di repository tapi **tidak pernah dipanggil**

Task:

- ☐ [FE] **chat_notifier.dart** → **build()**: Saat inisialisasi, panggil `getSessions()`. Jika ada session, ambil session terbaru (pertama dari list yang diurutkan `updatedAt desc`), lalu panggil `getMessages(sessionId)` untuk load history.
- ☐ [FE] **chat_notifier.dart**: Simpan `sessionId` saat ini di state agar saat kirim pesan berikutnya, `sessionId` ikut dikirim (bukan buat session baru)

C2. 🟡 Multi-session: Backend support tapi UI tidak ada pilihan session

Masalah:

- User sekarang memiliki beberapa chat session (tiap kali "Mulai Obrolan Baru" → session baru di backend)
- Tapi di FE tidak ada UI untuk melihat/memilih session lama
- User bingung karena session count di profil bertambah tapi chat selalu kosong

Keputusan:

Tambahkan fitur simple session list di chat screen — berupa drawer atau bottom sheet yang menampilkan daftar session. Tap = load history session itu.

Task:

- ☐ [FE] **ChatScreen**: Tambahkan tombol (misal icon  atau  di AppBar) yang membuka **daftar sesi chat**
 - ☐ [FE] **Buat widget ChatSessionList**: Tampilkan list session dari `getSessions()` — tiap item menunjukkan `title` + `updatedAt` + jumlah pesan
 - ☐ [FE] **ChatScreen**: Saat user tap session dari list → load messages session itu via `getMessages(sessionId)` dan set `_sessionId` ke session yang dipilih
 - ☐ [FE] **ChatScreen**: Tombol "Mulai Obrolan Baru" (yang sudah ada) → clear messages + set `sessionId` = null (ini sudah benar)
-

C3. Chat: "Mulai Obrolan Baru" tidak benar-benar clear session

Masalah:

- Tombol refresh di chat hanya clear `_messages` dan `_sessionId` di local state
- Session lama tetap ada di backend (tidak dihapus)
- Ini sebenarnya OK (session lama bisa diakses lagi) tapi user mungkin bingung

Task:

- ☐ [FE] **ChatScreen**: Setelah task C2 selesai (ada session list), pastikan "Mulai Obrolan Baru" benar-benar bikin session baru di next message (`sessionId=null` → backend auto-create).  Ini sudah benar di kode saat ini.
 - ☐ [BE] (opsional) Tambahkan endpoint `DELETE /api/chat/:id` untuk hapus session jika diperlukan nanti
-

D. Profile & Logout

D1. Logout: Layar blank

Masalah:

- User melaporkan: "ketika di profil mau logout layar malah blank"
- Dari audit kode, logout flow:
 1. `authNotifier.logout()` → clear semua storage (`_storage.deleteAll()`)
 2. Set state → `AuthStatus.unauthenticated`
 3. GoRouter redirect logic mendeteksi `unauthenticated` → redirect ke `/welcome`

Kemungkinan penyebab:

1. `deleteAll()` menghapus SEMUA secure storage termasuk hal yang mungkin masih dibutuhkan
2. Race condition: state berubah ke `unauthenticated` SEBELUM storage selesai di-clear → router redirect terjadi tapi screen belum ready
3. GoRouter redirect rebuild terjadi saat widget tree sedang dispose → blank screen

Task:

- ☐ [FE] **auth_notifier.dart** → **logout()**: Debug race condition — pastikan `await tokenService.clearAll()` selesai **sebelum** state diubah ke `unauthenticated`
 - ☐ [FE] **app_router.dart** → **redirect**: Tambahkan null check / guard agar redirect tidak terjadi saat context sedang dispose. Cek apakah `mounted` atau equivalent
 - ☐ [FE] **token_service.dart**: Ganti `deleteAll()` menjadi `delete(key: tokenKey)` saja — jangan hapus semua storage, cukup hapus token
 - ☐ [FE] Test logout flow: dari profil → tap logout → konfirmasi → harus muncul WelcomeScreen tanpa blank flash
-

D2. Profile: Stats hardcoded

Masalah:

- "Sesi Chat" selalu tampil **"1"** (hardcoded)

- "Akurasi" selalu tampil **"89%"** (hardcoded)
- Total Scan diambil dari scan history count (benar), tapi 2 stat lainnya fake

Task:

- ☐ [FE] **ProfileScreen**: Ambil sesi chat count dari `user.totalChats` (setelah task A4 selesai, BE sudah return `_count.chatSessions`)
- ☐ [FE] **ProfileScreen**: Untuk "Akurasi" — hitung dari data feedback scan. Atau lebih simpel: **hapus metric "Akurasi"** karena data tidak cukup untuk dihitung. Ganti dengan informasi lain atau hilangkan saja.

D3. 🟡 Profile: Edit hanya local, tidak persist ke API

Masalah:

- FE punya "Edit Profil" yang bisa ubah `aboutMe` — tapi **hanya update Riverpod state**, tidak panggil API
- Kalau user restart app, perubahan hilang

Keputusan:

Untuk MVP, **hapus tombol Edit Profil** atau buat readonly. Alasannya: tidak ada endpoint `PATCH /api/auth/profile` di backend, dan menambahnya = scope creep.

Task:

- ☐ [FE] **ProfileScreen**: Jadikan semua info profil **read-only** (hapus tombol "Edit Profil"). Atau:
- ☐ [BE] (opsional) Tambahkan endpoint `PATCH /api/auth/profile` yang bisa update name, phone, address, aboutMe → lalu FE bisa panggil ini

E. Security & Backend Hygiene

E1. 🔴 JWT Strategy mengembalikan password hash ke semua controller

Masalah:

- `jwt.strategy.ts` → `validate()` : Query user dari DB **tanpa select**, return full object termasuk `password` (hashed bcrypt)
- Semua controller yang pakai `@CurrentUser()` menerima object dengan password hash
- Kalau ada endpoint yang return `user` object langsung → password hash bocor ke client

Task:

- ☐ [BE] `jwt.strategy.ts` → `validate()`: Tambahkan `select` untuk exclude password:


```
select: { id: true, name: true, email: true, phone: true, address: true, aboutMe: true, createdAt: true }
```

Atau: pakai `delete user.password` sebelum return

E2. 🟡 `/api/test-gemini` endpoint tidak dilindungi auth

Masalah:

- `app.controller.ts` punya `GET /api/test-gemini?pesan=...` yang langsung memanggil Gemini API
- **Tidak ada** `@UseGuards(JwtAuthGuard)` — siapapun bisa akses dan membakar API credits

Task:

- ☐ [BE] `app.controller.ts`: Hapus endpoint `/api/test-gemini` seluruhnya (ini cuma untuk debugging, tidak perlu di production). Atau tambahkan `@UseGuards(JwtAuthGuard)`.

E3. 🟡 JWT Secret masih placeholder

Masalah:

- `.env` berisi `JWT_SECRET="ganti-dengan-random-string-panjang-minimal-32-karakter"` — literal placeholder text
- Ini berarti token bisa diprediksi/forged

Task:

- ☐ [BE] `.env` : Generate random JWT secret yang proper (minimal 32 karakter random). Contoh: `openssl rand -base64 32`
-

E4. 🟡 JWT_EXPIRES_IN di `.env` diabaikan

Masalah:

- `.env` punya `JWT_EXPIRES_IN="7d"` tapi `auth.module.ts` hardcode `expiresIn: '1d'`
- Config dari `.env` tidak terbaca

Task:

- ☐ [BE] `auth.module.ts` → `JwtModule.registerAsync()`: Ubah `expiresIn` agar baca dari `ConfigService`:

```
expiresIn: config.get<string>('JWT_EXPIRES_IN', '7d')
```

E5. 🟢 `getScanById` throw generic Error → 500 bukan 404

Masalah:

- Kalau scan tidak ditemukan, service throw `Error('Scan not found')` bukan `NotFoundException`
- Client dapat 500 Internal Server Error alih-alih 404 Not Found

Task:

- ☐ [BE] `scan.service.ts` → `getScanById()`: Ganti `throw new Error(...)` menjadi `throw new NotFoundException('Scan not found')`
-

F. Minor / Polish

F1. 🟢 Home screen "Tanya FarmerBot" button tidak navigate

Masalah (versi audit awal):

- Button "Tanya FarmerBot" di home screen punya komentar `// Navigate to chat tab` tapi logic kosong

Task:

- ☐ [FE] **HomeScreen**: Pastikan tombol "Tanya FarmerBot" navigate ke chat tab (via `context.go('/chat')` atau method yang sesuai dengan `GoRouter`)
-

F2. 🟢 "Lihat semua" scan history di Home = no-op

Masalah:

- Tombol "Lihat Semua →" pada section scan terakhir di `HomeScreen` punya `onPressed: () {}` — tidak melakukan apa-apa

Task:

- ☐ [FE] **HomeScreen**: Hubungkan tombol ke halaman history scan (navigate ke screen riwayat lengkap)
-

F3. 🟢 Chat imageUrl disimpan di DB tapi tidak dikirim ke Gemini

Masalah:

- SendMessageDto menerima imageUrl? dan disimpan ke ChatMessage di DB
- Tapi chat.service.ts → sendMessage() hanya mengirim dto.message (text) ke Gemini, **bukan gambar**
- Kalau user kirim foto di chat → foto tersimpan tapi Gemini tidak melihatnya

Task:

- ☐ [BE] (opsional — boleh skip untuk MVP) chat.service.ts: Kalau imageUrl ada, kirim juga ke Gemini sebagai inline image data (mirip scan analyze). Atau: buat catatan bahwa fitur kirim foto di chat belum supported.

F4. 🟢 Scan history tile import path mungkin salah

Masalah (dari audit):

- scan_repository.dart import path ../../../../../../core/network/... mungkin tidak sesuai (4 level dari file yang cuma 3 level deep)

Task:

- ☐ [FE] Verifikasi semua import path di scan_repository.dart — pastikan build Flutter sukses tanpa error

F5. 🟢 Gemini chat response mungkin JSON-wrapped

Masalah:

- System instruction meminta chat response dalam JSON: { "reply": "..." }
- Kalau Gemini patuh → chatService.sendMessage return { sessionId, reply: '{ "reply": "..." }' } — double wrapped
- FE akan tampilkan JSON mentah di chat bubble

Task:

- ☐ Sudah ter-cover oleh **Task B1** (hapus instruksi JSON dari system instruction). Setelah B1 selesai, chat response akan plain text.

📊 Ringkasan Prioritas

Harus dikerjakan PERTAMA (blocking issues):

#	Task	Sisi	Effort
A1	Login: ubah label Username → Email	FE	15 menit
A2	Register: hapus Username, tambah Email	FE + BE	30 menit
B1	Gemini: hapus instruksi JSON, konsistenkan ke markdown	BE	30 menit
B2	Scan parsing: rewrite jadi lebih robust	BE	1 jam
C1	Chat: load history saat buka app	FE	45 menit
D1	Logout: fix blank screen	FE	30 menit
E1	JWT: jangan return password hash	BE	10 menit

Dikerjakan setelahnya:

#	Task	Sisi	Effort
A4	Profile stats dari backend _count	BE + FE	20 menit
B3	Severity matching flexible di FE	FE + BE	20 menit
B4	Static file serving untuk uploads	BE	15 menit
C2	Chat session list UI	FE	1-2 jam
D2	Profile stats: hapus hardcoded	FE	15 menit
E2	Hapus /api/test-gemini	BE	5 menit
E3	Generate JWT secret proper	BE	5 menit
E4	Baca JWT_EXPIRES_IN dari env	BE	5 menit
E5	Scan not found → 404	BE	5 menit

Bisa di-skip untuk MVP:

#	Task	Alasan skip
C3	Delete session endpoint	User bisa ignore session lama
D3	Edit profil persist ke API	Read-only profil cukup untuk MVP
F1	Tanya FarmerBot button	Sudah ada tab Chat di bottom nav
F3	Kirim foto di chat ke Gemini	Fitur scan sudah cover ini

 Checklist Pengerjaan

```
Batch 1 – Critical Fixes (estimasi 3-4 jam):
├ [ ] A1 Login label Username → Email
├ [ ] A2 Register: hapus Username, tambah Email
├ [ ] B1 Gemini: konsistenkan format ke markdown
├ [ ] B2 Scan parsing robust
├ [ ] C1 Chat: load history saat init
├ [ ] D1 Logout: fix blank screen
└ [ ] E1 JWT: exclude password hash

Batch 2 – Important Fixes (estimasi 2-3 jam):
├ [ ] A4 Profile stats dari _count
├ [ ] B3 Severity matching flexible
├ [ ] B4 Static file serving uploads
├ [ ] C2 Chat session list UI
├ [ ] D2 Hapus stats hardcoded
├ [ ] E2 Hapus /api/test-gemini
├ [ ] E3 JWT secret proper
├ [ ] E4 JWT_EXPIRES_IN dari env
└ [ ] E5 Scan 404 bukan 500

Batch 3 – Polish (opsional):
├ [ ] F1 Tanya FarmerBot navigate
├ [ ] F2 Lihat semua scan history
├ [ ] F4 Verify import paths
└ [ ] D3 Edit profil (skip atau readonly)
```

Catatan: Kerjakan Batch 1 **satu per satu**, test setelah tiap task selesai.
Jangan kerjakan semuanya lalu test di akhir — risikonya bug baru menumpuk.